

ACQUISITIONOS · PAYMENT CONFIGURATION

Stripe Setup Guide

Complete beginner-to-production walkthrough for Stripe in AcquisitionOS — account, products, recurring prices, GST-inclusive amounts, checkout, webhooks, verification, and Test → Live migration.

[Hosted Checkout](#)[Recurring Prices](#)[Webhooks](#)[GST-inclusive amounts](#)[Billing Portal](#)

Version 1.0 · September 23, 2026 · Documentation only — verified against the current codebase

This guide is a complete, beginner-to-production walkthrough for configuring Stripe for the **current** AcquisitionOS codebase. Every environment variable name, endpoint URL, webhook event, and amount documented here was verified directly against the source code (`src/lib/payment-service.ts` , `src/lib/payments/*` , `src/app/api/payments/webhook/stripe/route.ts` , and related files). Where the Stripe Dashboard UI may differ across versions, instructions are marked **UI may vary by provider dashboard version** and the conceptual navigation is also given.

Part 1 — How AcquisitionOS Uses Stripe (Read This First)

Before touching the Stripe Dashboard, understand exactly what the application expects. AcquisitionOS integrates Stripe through **hosted Stripe Checkout** (not embedded forms, not raw PaymentIntents created by the app). The flow is:

1. The user clicks a plan button in the app (Pricing page or Settings → Billing).
2. The app calls `POST /api/payments/create-checkout-session` , which creates a real Stripe Checkout Session with `mode: 'subscription'` (plans) or `mode: 'payment'` (credit add-ons) and **one line item referencing an env-configured Stripe Price ID**. No amount is ever hardcoded — the Stripe Price object is the single source of truth for the charged amount.
3. The browser redirects to Stripe's hosted payment page (`window.location.href = data.url`).
4. After payment, Stripe redirects back to `/dashboard?payment=success&session_id={CHECKOUT_SESSION_ID}` .
5. **The Stripe webhook (`/api/payments/webhook/stripe`) is the ONLY path that activates the subscription and grants credits in the database.** A secondary reconciliation route (`/api/payments/verify-session`) confirms the session after redirect and activates if the webhook has not yet arrived.
6. The app generates its **own** invoice (numbered, GST-itemized, PDF + email) on every activation — it does not depend on Stripe Invoices for receipts.

Key architectural facts that drive every configuration step in this guide:

Fact	Consequence for setup
Checkout uses <code>line_items: [{ price: <Price ID>, quantity: 1 }]</code>	You MUST create real recurring Prices and copy their IDs into env vars
The Price object amount is authoritative	Create each Price at the GST-inclusive amount so the charge matches the app's displayed total (base + 18% GST)
Webhook drives activation	The webhook endpoint and signing secret are mandatory; production rejects unsigned/placeholder secrets
Idempotency via the <code>PaymentWebhook</code> table	Replaying/duplicating webhooks is safe — events are deduplicated by event ID
Currency	Stripe checkout in this app is USD-typed by the provider adapter; create Prices in INR for Indian users (GST math) and/or USD for international. Whatever currency the Price carries is what the session charges
Missing Price ID env var	The plan/cycle button shows "Coming Soon" in the UI and checkout is never initiated (no error shown to users)

Part 2 — Create / Log In to Your Stripe Account

1. Go to <https://dashboard.stripe.com/register>.
2. Sign up with your business email (the same identity you use for AcquisitionOS administration is recommended).
3. Verify your email address and enable two-factor authentication (recommended).
4. If you already have a Stripe account, log in at <https://dashboard.stripe.com/login> and decide whether to use the existing account or create a **new account** inside it for AcquisitionOS (Accounts menu → *New account*). Keeping AcquisitionOS in its own Stripe account simplifies keys, webhooks, and reporting.

Part 3 — Create / Configure the Business Profile

Stripe requires business details before you can go live (and to accept real payments in Test Mode you can skip most of this).

1. Open **Settings** → **Account settings** → **Business details** (UI may vary).
2. Fill in: legal business name, business structure (individual/LLC/Pvt Ltd), registered address, phone, and industry (SaaS / Software).
3. Add a payout bank account: **Settings** → **Bank accounts and schedules**. Payouts cannot be enabled without it.
4. Add public business details shown to customers: **Settings** → **Public details** (statement descriptor — e.g. `ACQUISITIONOS`, customer support email, and logo). The statement descriptor appears on the customer's card statement.
5. For India-based businesses, also review **Settings** → **Tax details** (GSTIN registration affects how Stripe reports your payouts; this is separate from the app's own GST-invoicing, which is computed in-app).

Part 4 — Test Mode vs Live Mode

Stripe has two isolated environments, toggled from the Dashboard top bar:

Mode	Key prefixes	What works	Use
Test	<code>sk_test_...</code> , <code>pk_test_...</code> , <code>price_...</code> (test objects)	Simulated payments with test cards (e.g. <code>4242 4242 4242 4242</code>), fake webhooks	All setup, integration verification, rehearsal
Live	<code>sk_live_...</code> , <code>pk_live_...</code>	Real cards, real money	Only after the Test checklist passes

AcquisitionOS **auto-detects the mode from the key prefix**. `src/lib/payment-service.ts` and `src/lib/payments/provider-status` route report `mode: 'test'` when `STRIPE_SECRET_KEY` starts with `sk_test_`, and `mode: 'live'` otherwise. You do not need to configure the mode anywhere — the prefix is the switch.

Rule: products/prices/webhooks are separate objects per mode. Everything you create in Test Mode must be re-created in Live Mode, and each mode has its **own** Price IDs and webhook signing secret.

Part 5 — API Keys

Location: **Developers** → **API keys** (UI may vary); conceptual path: Dashboard home → gear/Developers → API keys).

5.1 Publishable key

- Identifier starts with `pk_test_` / `pk_live_`. Safe to expose in the browser.
- AcquisitionOS env var (client-facing): `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`
- A server-side alias `STRIPE_PUBLISHABLE_KEY` is also recognized by configuration validation (`src/lib/env-validation.ts`). Set at least one; the `NEXT_PUBLIC_` one is what `provider-status` surfaces to the UI.
- This key is displayed by `GET /api/payments/provider-status` (public endpoint) — it is designed to be public.

5.2 Secret key

- Identifier starts with `sk_test_` / `sk_live_`. **Never expose it** in the browser, git, logs, or client bundles.
- AcquisitionOS env var: `STRIPE_SECRET_KEY`

- Where it is consumed (verified in code): every Stripe SDK instantiation — `payment-service.ts` (`getStripeInstance()`), `stripe-portal-service.ts`, `refund-service.ts`, `invoice-pdf-service.ts`, the webhook route, and provider availability checks (`!!process.env.STRIPE_SECRET_KEY` gates all Stripe checkout).

Copy the keys now (reveal + copy button next to each). You will paste them into your environment/secrets store in Part 9.

Part 6 — Product & Price Model Used by AcquisitionOS

Stripe objects map to AcquisitionOS plans like this:

AcquisitionOS plan	Stripe Product you create	Stripe Prices you create	Charged amount to enter
Free	(none — never charged)	(none)	n/a
Starter	AcquisitionOS Starter	Starter Monthly + Starter Yearly (recurring)	see Part 7
Pro	AcquisitionOS Pro	Pro Monthly + Pro Yearly (recurring)	see Part 7
Elite	AcquisitionOS Elite	Elite Monthly + Elite Yearly (recurring)	see Part 7
Credit add-on 100 / 500 / 1000	AcquisitionOS Credit Pack 100/500/1000	One-time Price each	see Part 8

- **The Free plan needs no Stripe object.** It is granted in-app (50 credits/month) and never touches a gateway.
- Coupons: the app has its own coupon system (`Coupon` table, `/api/payments/validate-coupon`). On Stripe checkout a user-entered coupon code simply enables `allow_promotion_codes: true` on the session, so you may *optionally* also create Stripe Promotion Codes; the webhook reconciles the final amount either way.

Part 7 — Create Products & Recurring Prices (Plans)

UI may vary by provider dashboard version — conceptual navigation: **Dashboard** → **Product catalog** → **Add product**.

For **each** product below, repeat these steps:

1. **Product name / description** (exact values recommended):

Product name	Product description (recommended)
AcquisitionOS Starter	Starter plan: 150 credits/month, 25 lead discoveries/month
AcquisitionOS Pro	Pro plan: 750 credits/month, unlimited lead discovery
AcquisitionOS Elite	Elite plan: 2,000 credits/month, unlimited lead discovery, all features

1. Add **two recurring prices** to each product — one monthly, one yearly:

Price	Amount to enter	Currency	Billing interval
Starter Monthly	₹589	INR	Every 1 month
Starter Yearly	₹5,899	INR	Every 1 year
Pro Monthly	₹1,887	INR	Every 1 month
Pro Yearly	₹17,699	INR	Every 1 year
Elite Monthly	₹6,135	INR	Every 1 month
Elite Yearly	₹53,099	INR	Every 1 year

1. **Why these amounts (GST):** the app prices plans in base amounts and displays 18% GST on top — e.g. Pro Monthly shows base ₹1,599 + GST ₹288 = **total ₹1,887**. Because the Stripe Price is the authoritative charged amount, create each Price at the **GST-inclusive total** above so the card charge equals the app's displayed total. The full base/GST/total table:

Plan	Cycle	Base (₹)	GST 18% (₹)	Total = Price amount (₹)
Starter	Monthly	499	90	589
Starter	Yearly	4,999	900	5,899
Pro	Monthly	1,599	288	1,887
Pro	Yearly	14,999	2,700	17,699
Elite	Monthly	5,199	936	6,135
Elite	Yearly	44,999	8,100	53,099

If you prefer USD pricing for international customers, create equivalent USD Prices (app display values: 6 / 60 Starter, 19 / 180 Pro, 63 / 540 Elite) and note that the session will charge in whatever currency the referenced Price carries.

1. After creating each Price, open it and copy the **Price ID** (`price_...`). You will map six recurring Price IDs in Part 9.

Do not enable per-Price "automatic tax" (Stripe Tax) unless you also add `automatic_tax` to the app's session creation (it is not in the current code). Tax is already baked into the amounts above.

Part 8 — Credit Add-On Products (One-Time Prices)

AcquisitionOS sells three one-time credit packs (catalog verified in `src/app/api/payments/credit-addons/route.ts`):

Pack ID (in-app)	Credits	Base (₹)	GST 18% (₹)	Total = Price amount (₹)	Price type
<code>credits_100</code>	100	499	90	589	One time
<code>credits_500</code>	500	1,999	360	2,359	One time
<code>credits_1000</code>	1,000	3,499	630	4,129	One time

Steps (per pack): **Product catalog** → **Add product** → name `AcquisitionOS Credit Pack 100` (and 500 / 1,000) → description "One-time top-up of N lead-generation credits (never expires)" → add a **one-time** Price with the amount above, currency INR → copy each `price_...` ID. No billing interval. These checkouts are created with `mode: 'payment'` and metadata `type: 'credits'`, and fulfillment routes through the app's credit-addon logic — nothing else to configure on Stripe's side.

Part 9 — Copy Each Price ID Into the Exact Environment Variables

AcquisitionOS resolves plan Price IDs **only from environment variables** (never hardcoded). The resolver (`resolvePlanPriceId` in `src/lib/payment-service.ts`) accepts four naming conventions **in this order** and uses the first real value (must start with `price_`):

- `STRIPE_{PLAN}_{CYCLE}_PRICE_ID`
- `STRIPE_PRICE_ID_{PLAN}_{CYCLE}`
- `STRIPE_PRICE_{PLAN}_{CYCLE}_ID` ← **canonical (recommended)**
- `STRIPE_PRICE_{PLAN}_{CYCLE}`

Recommended mapping (set all six):

Plan • Cycle	Recommended env var (canonical)	Also accepted (aliases)
--------------	---------------------------------	-------------------------

Starter · Monthly	<code>STRIPE_PRICE_STARTER_MONTHLY_ID</code>	<code>STRIPE_STARTER_MONTHLY_PRICE_ID</code> , <code>STRIPE_PRICE_ID_STARTER_MONTHLY</code> , <code>STRIPE_PRICE_STARTER_MONTHLY</code>
Starter · Yearly	<code>STRIPE_PRICE_STARTER_YEARLY_ID</code>	<code>STRIPE_STARTER_YEARLY_PRICE_ID</code> , <code>STRIPE_PRICE_ID_STARTER_YEARLY</code> , <code>STRIPE_PRICE_STARTER_YEARLY</code>
Pro · Monthly	<code>STRIPE_PRICE_PRO_MONTHLY_ID</code>	<code>STRIPE_PRO_MONTHLY_PRICE_ID</code> , <code>STRIPE_PRICE_ID_PRO_MONTHLY</code> , <code>STRIPE_PRICE_PRO_MONTHLY</code>
Pro · Yearly	<code>STRIPE_PRICE_PRO_YEARLY_ID</code>	<code>STRIPE_PRO_YEARLY_PRICE_ID</code> , <code>STRIPE_PRICE_ID_PRO_YEARLY</code> , <code>STRIPE_PRICE_PRO_YEARLY</code>
Elite · Monthly	<code>STRIPE_PRICE_ELITE_MONTHLY_ID</code>	<code>STRIPE_ELITE_MONTHLY_PRICE_ID</code> , <code>STRIPE_PRICE_ID_ELITE_MONTHLY</code> , <code>STRIPE_PRICE_ELITE_MONTHLY</code>
Elite · Yearly	<code>STRIPE_PRICE_ELITE_YEARLY_ID</code>	<code>STRIPE_ELITE_YEARLY_PRICE_ID</code> , <code>STRIPE_PRICE_ID_ELITE_YEARLY</code> , <code>STRIPE_PRICE_ELITE_YEARLY</code>

Credit pack mapping (names are fixed — no aliases):

Pack	Env var (exact)
100 credits	<code>STRIPE_PRICE_CREDITS_100_ID</code>
500 credits	<code>STRIPE_PRICE_CREDITS_500_ID</code>
1,000 credits	<code>STRIPE_PRICE_CREDITS_1000_ID</code>

The four canonical variables previously documented for the Sep-2026 pricing update (`STRIPE_PRICE_STARTER_MONTHLY_ID` , `STRIPE_PRICE_STARTER_YEARLY_ID` , `STRIPE_PRICE_PRO_YEARLY_ID` , `STRIPE_PRICE_ELITE_YEARLY_ID`) are exactly convention #3 above — the recommended names in this table are the same family, extended to all six plan/cycle combinations for consistency.

Part 10 — Checkout Configuration (What the App Creates)

For reference — and for verifying behavior — these are the exact Checkout Session parameters the app sends (from `createStripeCheckoutSession`):

- `mode`: `'subscription'` (plans) / `'payment'` (credit packs)
- `payment_method_types`: `['card']`
- `line_items`: `[{ price: <env Price ID>, quantity: 1 }]`
- `success_url`: `{APP_URL}/dashboard?payment=success&session_id={CHECKOUT_SESSION_ID}` (overridable via `STRIPE_SUCCESS_URL`)
- `cancel_url` : `{APP_URL}/dashboard?payment=cancelled` (overridable via `STRIPE_CANCEL_URL`)
- `metadata` : `userId` , `user_id` , `planName` , `plan` , `billingCycle` , `order_id` , `orderId` , `couponCode` (credit packs: `userId` , `type`: `'credits'` , `creditAmount` , `addonId` , `orderId`)
- `subscription_data.metadata` : same plan context, so `customer.subscription.*` events can be attributed
- `client_reference_id` : the app user ID
- `allow_promotion_codes`: `true` (only when a coupon code was entered)

Nothing to configure in Stripe for this — hosted Checkout is enabled by default on every account. The only inputs are the Price IDs (Part 9) and redirect URLs (optional env overrides).

Part 11 — Customer Configuration

AcquisitionOS creates/reuses Stripe Customers via `ensureStripeCustomer()` in `src/lib/stripe-portal-service.ts` (email + name, idempotent per user). The resulting `stripeCustomerId` is stored on the app's `Subscription` record. This happens

automatically (primarily for the Billing Portal) — **no manual Stripe configuration is required**. Optionally, in the Dashboard you can set customer-facing defaults under **Settings → Public details** (support email/logo shown on invoices/receipts).

Part 12 — Subscription Configuration

- Subscriptions are created by Stripe automatically from `mode: 'subscription'` Checkout Sessions; the app stores `stripeSubscriptionId` + `stripeCustomerId` on its `Subscription` record and keeps `currentPeriodStart/End` in sync from webhook events (`customer.subscription.updated` , `invoice.paid`).
- Cancellation is app-initiated (`POST /api/payments/cancel` → `cancelAtPeriodEnd: true` on the app record; the Stripe-side cancellation is confirmed via `customer.subscription.updated/deleted` webhooks).
- No billing-cycle automation to configure in Stripe.** Renewal handling in the app: for Stripe subscriptions, renewals arrive as `invoice.paid` / `invoice.payment_succeeded` and reset monthly credits with period-idempotency (a `period_start` match check prevents double resets). (For the Razorpay one-time-order model, an end-of-period cron does the equivalent.)

Part 13 — PaymentIntent, Invoice, and Refund Notes

- PaymentIntent:** the app never creates PaymentIntents directly; Checkout creates them under the hood. The webhook handles `payment_intent.succeeded` as a fallback activator for orders found by `providerPaymentId` — informational only.
- Stripe Invoices:** for subscriptions Stripe generates its own invoices, and the app listens to `invoice.paid` / `invoice.payment_succeeded` for renewals. However, the customer-facing invoice/receipt in AcquisitionOS is the app's **own** numbered GST invoice (PDF + email), generated on every activation — you do not need to configure Stripe Invoicing settings beyond defaults.
- Refunds:** issue refunds from the Stripe Dashboard (**Payments → click payment → Refund**) or via the app's super-admin endpoint `POST /api/admin/refund` (which calls Stripe's Refund API). The `charge.refunded` webhook reverses credits and downgrades the plan on full refunds. Partial refunds are supported. No Stripe-side configuration needed.

Part 14 — Create the Webhook Endpoint

The webhook is **mandatory** — without it no subscription activates.

- Endpoint URL (exact):** `https://<your-domain>/api/payments/webhook/stripe`
- For the current deployment that is: `https://acquisition.space-z.ai/api/payments/webhook/stripe`
- Use your production domain in Live Mode and the same (or a tunnel/dev URL) in Test Mode.
- Navigation: **Developers → Webhooks → Add endpoint** (`UI may vary` ; conceptual: **Dashboard → Developers → Webhooks → Add endpoint**).
- Paste the endpoint URL above.
- Select the events listed in Part 15.
- Click **Add endpoint**, then open the endpoint's detail page and click **Reveal** under *Signing secret*. Copy the `whsec_...` value → this is your `STRIPE_WEBHOOK_SECRET` env var. Test Mode and Live Mode each have their own secret.

Security behavior verified in code: the route validates every delivery's `stripe-signature` header via `constructEvent` using `STRIPE_WEBHOOK_SECRET`. Placeholder secrets (containing `YOUR` or the literal `whsec_YOUR_STRIPE_WEBHOOK_SECRET`) are rejected, and production demands a valid signature.

Part 15 — Required Webhook Events (Exact List)

Subscribe the endpoint to exactly these events (all are handled by the current code):

Event	What the app does
<code>checkout.session.completed</code>	Primary activation: marks the order completed, activates the subscription, grants plan credits (or fulfills a credit add-on), generates invoice + email
<code>checkout.session.expired</code>	Marks the pending order failed (user abandoned checkout)
<code>charge.refunded</code>	Reverses credits; downgrades plan on full refund
<code>charge.dispute.created</code>	Records a chargeback alert on the affected order
<code>customer.subscription.created</code>	Accepted and recorded; no dedicated logic
<code>customer.subscription.updated</code>	Syncs Stripe status → app status (<code>active</code> , <code>trialing</code> , <code>past_due</code> , <code>canceled</code> , <code>unpaid → expired</code>), period dates, <code>cancelAtPeriodEnd</code>
<code>customer.subscription.deleted</code>	Marks the subscription canceled/expired in the app
<code>invoice.payment_succeeded</code>	Renewal payment: extends period, resets monthly credits (period-idempotent)
<code>invoice.paid</code>	Duplicate safety net for the same renewal (skips if already processed for the period)
<code>invoice.payment_failed</code>	Marks the subscription <code>past_due</code> , logs the failure, triggers recovery/dunning UI
<code>payment_intent.succeeded</code>	Fallback activation path for orders matched by PaymentIntent ID

(Unknown/other events are safely marked processed and ignored — subscribing to extra events is harmless.)

Part 16 — Test Webhook Delivery

Fastest end-to-end test (Stripe CLI, `UI may vary` — CLI works for both modes):

```
stripe login
stripe listen --forward-to localhost:3000/api/payments/webhook/stripe
stripe trigger checkout.session.completed
```

- `stripe listen` prints a temporary `whsec_...` — use it as `STRIPE_WEBHOOK_SECRET` while developing locally.
- To test a *real* checkout: create a Test-Mode Price (Part 7), run a checkout with card `4242 4242 4242 4242`, any future expiry, any CVC, and watch the forwarded event arrive.
- Dashboard-only alternative: **Developers** → **Webhooks** → **your endpoint** → **Send test webhook** and pick each event from Part 15.

Part 17 — Inspect Webhook Failures

- **Stripe side:** **Developers** → **Webhooks** → **endpoint** → **deliveries tab** — shows every attempt, response code, and the full request/response. A `400` from our endpoint means signature verification failed (wrong secret or modified body).
- **App side:** every delivery is recorded in the `PaymentWebhook` table (`eventId` , `eventType` , `processed` , `processingError`) and surfaced in the admin UI via `GET /api/admin/billing/webhooks` . Failed events are also logged to the billing audit log.
- Quick app-side probe: `GET /api/payments/provider-status` (should show `stripe.available = true`), then attempt a Test-Mode checkout.

Part 18 — Replay a Webhook Safely

Replays are **safe by design**: the app deduplicates by event ID (`PaymentWebhook.eventId` unique) and re-checks that the order is still `pending` before acting, inside an atomic transaction.

1. In the Dashboard, open the failed delivery → ... → **Resend** (or select multiple → Resend). `UI may vary`.
2. Or use the CLI: `stripe events resend evt_<id>`.
3. The app also exposes an operator route, `POST /api/payments/webhook-replay`, for internal replay tooling.
4. Re-verify the order reached `completed` in the admin payment history.

Part 19 — Verify Successful / Failed / Pending Payments

State	Stripe side	App side (what to check)
Success	Payment shows <i>Succeeded</i> ; Checkout session <code>complete</code>	<code>PaymentOrder.status = completed</code> ; <code>Subscription.status = active</code> with correct plan + period; user credits increased by plan amount (50/150/750/2000); invoice row + PDF created; success modal shown on <code>/dashboard?payment=success...</code>
Failed	Checkout session <code>expired</code> , or card declined on the hosted page	<code>PaymentOrder.status = failed</code> ; <code>checkout.session.expired</code> webhook recorded; failed-payment modal / banner shown to the user
Pending / processing	Payment <code>processing</code> (async methods) or session not yet complete	Order stays <code>pending</code> — the webhook or <code>POST /api/payments/verify-session</code> (called after redirect) will resolve it; the cron job <code>/api/cron/payment-reconciliation</code> also reconciles stale pending orders; users see a "confirming payment" state
Refund	Refund row on the payment	<code>charge.refunded</code> webhook processed; credits reversed (<code>CreditsLedger</code> negative entry); full refund downgrades the plan

Part 20 — Verify Every Price ID Is Correctly Mapped

1. `GET /api/payments/provider-status` (public, no auth) → check `planAvailability.<plan>.<cycle>.stripe = true` for all six plan/cycle combinations after configuring keys + Price IDs. Before Price IDs exist, `stripe` is `false` and the UI shows **Coming Soon** — this is the intended gate.
2. Attempt a real Test-Mode checkout for each plan/cycle from Settings → Billing (use the test card above). The charged amount on the hosted page must equal the GST-inclusive total from Part 7.
3. Buy each credit pack in Test Mode; verify the credit balance increases by exactly 100 / 500 / 1000.
4. Server logs show the resolver's exact env-var names on failure: a missing ID produces the error `"Stripe price ID for <plan> (<cycle>) is not configured. Set one of: ..."` — the message lists every accepted variable name.

Part 21 — Test Mode → Live Mode Migration Checklist

- [] Business profile activated; payouts enabled (no blocking requirements under **Home**)
- [] Toggle Dashboard to **Live mode**
- [] Create all 3 plan products + 3 credit-pack products in Live Mode (Products are mode-scoped)
- [] Copy the **Live** Price IDs into the six plan env vars + three credit-pack env vars (Part 9)
- [] Replace `STRIPE_SECRET_KEY` with the `sk_live_...` key
- [] Replace `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` with the `pk_live_...` key
- [] Create a **Live-mode webhook endpoint** (same URL) subscribed to the Part 15 events
- [] Set `STRIPE_WEBHOOK_SECRET` to the **Live** endpoint's `whsec_...`

- [] `GET /api/payments/provider-status` shows `stripe.mode = 'live'` and all plan availability flags true
- [] Run one real live transaction end-to-end (then refund it if it was a self-test) and verify webhook activation
- [] Confirm statement descriptor and public support details are set

Part 22 — Production Verification Checklist

- [] Webhook endpoint reachable over public HTTPS (no localhost) and returns 200 on test delivery
- [] `charge.refunded`, `invoice.paid`, and `customer.subscription.updated` observed at least once each (trigger in Test, or after first real events)
- [] Invoice PDF + email generated on a real activation (check the user's inbox and `/api/payments/invoices`)
- [] Billing Portal opens: Settings → Billing → *Manage billing* → `POST /api/payments/stripe-portal` (return URL defaults to `{APP_URL}/settings?tab=billing`, override via `STRIPE_PORTAL_RETURN_URL`)
- [] Cancellation path verified: cancel → `cancelAtPeriodEnd=true` → plan keeps working until period end
- [] Credits ledger (`CreditsLedger`) shows the correct positive entry on purchase and monthly reset behavior on renewal
- [] Cron routes reachable (with `CRON_SECRET`): `/api/cron/renew-subscriptions`, `/api/cron/payment-reconciliation`

Part 23 — Common Mistakes & Troubleshooting

Symptom	Cause	Fix
Plan buttons show "Coming Soon" even with keys set	The plan/cycle's Price ID env var is missing (availability = secret key AND Price ID present)	Set the exact env var(s) from Part 9; restart/redeploy
Checkout returns 500: <code>"Stripe price ID for ... is not configured. Set one of: ..."</code>	Same as above, surfaced server-side	The error lists every accepted variable name — set one
Webhook returns 400 <code>Invalid signature</code>	Wrong <code>STRIPE_WEBHOOK_SECRET</code> for that mode, or secret from the other endpoint	Copy the signing secret from the <i>correct</i> endpoint in the <i>correct</i> mode
Webhook returns 500 <code>Webhook verification not configured</code>	<code>STRIPE_WEBHOOK_SECRET</code> unset or still a placeholder in production	Set the real <code>whsec_...</code> ; placeholders are deliberately rejected
Payment taken but plan not activated	Webhook not configured / unreachable, or event not subscribed	Configure Part 14–15; then replay the event (Part 18) — idempotency makes this safe
Charged amount differs from the UI total	Price created at base amount (e.g. ₹1,599) instead of the GST-inclusive total (₹1,887)	Re-create the Price at the Part 7 amount (or enable Stripe Tax AND add <code>automatic_tax</code> in code — not currently implemented)
Webhook amount-mismatch rejections	Someone edited the Price after orders were created	Keep one Price per plan/cycle; if you must change pricing, create a <i>new</i> Price and update the env var
<code>payment_method_types</code> errors for UPI/wallets	App requests card-only Stripe sessions	Use Razorpay for UPI/netbanking; Stripe path is card-first
Double activation feared on retries	Safe by design	Not possible: <code>PaymentWebhook</code> dedup + atomic pending-check (documented behavior)
Coupons don't discount the checkout	Stripe-side promotion code doesn't exist for the entered code	Create the matching Promotion Code in Stripe, or rely on the app's own coupon validation for Razorpay-only flows
Test works, Live fails	Live Price IDs / webhook secret not updated	Redo Part 21 checklist

End of Stripe Setup Guide. Companion documents: `RAZORPAY-SETUP-GUIDE.md` (India-first gateway) and `PAYMENT-CONFIGURATION-MASTER.md` (cross-provider tables, env inventory, and the end-to-end checklist).

Configuration complete.

Every environment variable name, endpoint, webhook event, and amount in this guide was verified against the current AcquisitionOS source code. Configure objects in Test Mode first, verify the full checklist, then repeat in Live Mode.

Companion guides: Razorpay Setup Guide · Payment Configuration Master